Frappe / ERPNext Docker Deployment Tool

fda_script.py — Complete User Guide

Interactive CLI to deploy and manage Frappe/ERPNext on Docker

---

Document Contents

1. Overview & Prerequisites

What is this tool?

fda_script.py is an interactive, menu-driven command-line tool that guides you through every step of deploying and managing Frappe / ERPNext on Docker. You do not need to memorise Docker commands — the tool handles everything.

Prerequisites

- Python 3.8 or higher
- Linux / macOS (Docker Desktop) / WSL2 on Windows
- Internet access (for image pulls and Docker install)
- frappe_docker repository (the tool can clone it for you)
- Docker (the tool can install it for you via Option 1)

Gitops Directory

The tool stores all configuration in ~/gitops (your home directory). This folder contains .env files and resolved .yaml files for every deployed project. Never delete this folder — it is required to manage, restart, or remove running stacks.

2. How to Run the Tool

Step 1	Open a terminal on your server or WSL2 session.
Step 2	Navigate to the frappe_docker directory: cd ~/frappe_docker
Step 3	Run the tool: python3 fda_script.py
Step 4	The status panel loads, showing what is configured.
Step 5	Enter an option number and press Enter. Follow the prompts.
Step 6	Default values are shown in [brackets] — press Enter to accept.
Step 7	Press Q or type 'quit' to exit.

Status Panel

Every time the menu loads, a status panel shows the state of your environment:

- ✔ selected (green) — service is configured and active
- ✖ not selected (yellow) — not yet set up or missing
- Covers: OS Type, Docker, Repo, Gitops directory, Traefik, MariaDB, PostgreSQL

Maintenance Mode

Options 3–6 (Local Deploy group) are now fully active. Previously marked as maintenance mode, these options are no longer blocked and can be used normally.

3. Menu Structure at a Glance

GENERAL	
1	Install Docker & Docker Compose
2	Set active frappe_docker repo
LOCAL DEPLOY	
3	**Local deploy via pwd.yml**
4	**Local deploy status & diagnostics**
5	**Stop local deploy**
6	**Drop local deploy**
INFRASTRUCTURE	
7	Setup Traefik reverse proxy
8	Setup shared MariaDB database
9	Setup shared PostgreSQL database
10	Restart infrastructure servers
11	Drop infrastructure services
BENCH & SITES	
12	Deploy Frappe / ERPNext bench
13	Create a new site
14	Install an app on a site
15	Uninstall an app from a site
SITE OPERATIONS	
16	Migrate site
17	Clear site cache
18	**Set maintenance mode**
19	Enable / Disable scheduler
20	Drop / Delete a site
21	Restore site from backup
IMAGES	
22	View Docker images
23	Create custom image (apps.json + build)
24	Update image with latest git code
MANAGEMENT	
25	Update bench (pull → restart → migrate)
26	Stop a bench
27	View running containers
28	View container logs
29	Backup all sites
30	Push backup to S3 storage

31	Bench console (Python shell)
32	Clean volumes, networks & build cache

4. Detailed Option Reference

General

Option 1 — Install Docker & Docker Compose	
Description	Downloads and runs the official Docker install script (Linux), or installs via Homebrew (brew install --cask docker) / Docker Desktop download (macOS). Then installs the Docker Compose plugin.
When to Use	First-time setup on a new server before using any other option.
Steps	1. Select option 1 2. Confirm the install prompt (runs as root) 3. Docker engine is installed 4. Docker Compose plugin is installed automatically 5. Versions are verified and printed

Option 2 — Set Active frappe_docker Repo	
Description	Finds an existing frappe_docker directory or clones a fresh copy. Sets it as the working directory.
When to Use	Before any option that needs the repo. Run at the start of every session.
Steps	1. Select option 2 2. Tool searches ~/frappe_docker, /opt/frappe_docker, etc. 3. Choose an existing folder OR clone fresh from GitHub 4. ~/gitops directory is created automatically 5. All subsequent options now work from this directory
⚠ **The working directory is set for this session only.**	

Local Deploy ⚠

Option 3 — Local Deploy via pwd.yml	
Description	Updates pwd.yml with your chosen image, site name, passwords and app, then starts the full Frappe stack locally on port 8080. Blocked if a local deploy is already running. After deploy: auto-runs bench use + migrate. Uses sudo for all docker compose commands. Blocked if a local deploy is already running. After deploy: auto-runs bench use + migrate. Uses sudo for all docker compose commands.
When to Use	Local testing, development, QA demos — when you do not need Traefik or a domain.
Steps	1. Select option 3 2. View/choose from local Frappe images (or type a custom image name) 3. Enter site name (e.g. testing) 4. Enter MariaDB root password and Frappe admin password

	5. Enter app to install on first run (e.g. erpnext) 6. Choose Fresh (wipes volumes) or Update (keeps data) 7. Review the summary panel and confirm 8. Stack starts — access at http://localhost:8080 9. Site creation runs in background (~2–3 min on first run) 10. Guard: if a local deploy is already running, the option is blocked and user is redirected to Stop/Drop options 11. After successful deploy: automatically runs bench use <site> then bench --site <site> migrate (10-second delays between each) 12. pwd.yml restart policy is auto-fixed (restart: none → restart: "no") 13. All docker compose commands now use sudo 10. Guard: if a local deploy is already running, the option is blocked and user is redirected to Stop/Drop options 11. After successful deploy: automatically runs bench use <site> then bench --site <site> migrate (10-second delays between each) 12. pwd.yml restart policy is auto-fixed (restart: none → restart: "no") 13. All docker compose commands now use sudo

Option 4 — Local Deploy Status & Diagnostics	
Description	Shows all container states, create-site logs with highlighted errors, and backend error logs. Auto-diagnoses and offers fixes.
When to Use	After any local deployment to verify success. When browser shows errors.
Steps	1. Select option 4 2. Container status table is shown 3. create-site logs shown — errors highlighted in red/yellow 4. Backend last 10 log lines shown 5. If 'site already exists' detected → choose Fresh deploy or Migrate 6. If backend errors found → diagnosis and fix suggestion shown

Option 5 — Stop Local Deploy	
Description	Stops all pwd.yml containers. Volumes and all data are fully preserved.
When to Use	To free up resources while keeping data. Before suspending your machine.
Steps	1. Select option 5 2. Running containers are listed 3. Confirm stop 4. All containers stop — data intact 5. Restart later: docker compose -f pwd.yml start

Option 6 — Drop Local Deploy	
Description	Stops and removes the pwd.yml stack. Three levels: Stop only / Remove containers+network / Full wipe with volumes.

When to Use	Full cleanup of the local stack. Before a fresh deploy from zero.
Steps	1. Select option 6 2. Running containers are listed 3. Choose drop level: Level 1 — Stop only (data safe) Level 2 — Remove containers and network (data safe) Level 3 — Remove everything including volumes (ALL data deleted) 7. Confirm and execute 8. Orphan volumes from old projects are also cleaned up at Level 3

Live Infrastructure

Option 7 — Setup Traefik Reverse Proxy	
Description	Creates ~/gitops/traefik.env and starts Traefik. Traefik routes HTTP/HTTPS traffic to bench frontends by domain name.
When to Use	Production or staging setup. Must be running before deploying any bench with HTTPS.
Steps	1. Select option 7 2. Enter Traefik dashboard domain (e.g. traefik.mycompany.com) 3. Enter admin email 4. Enter dashboard password (hashed with openssl automatically) 5. Choose HTTPS with Let's Encrypt (recommended) or plain HTTP 6. Traefik starts on ports 80 and 443
⚠ **Domain DNS must point to this server for Let's Encrypt to work.**	

Option 8 — Setup Shared MariaDB Database	
Description	Creates ~/gitops/mariadb.env and starts a shared MariaDB 10.6 container on the internal mariadb-network.
When to Use	Before deploying any bench that uses MariaDB (the default database).
Steps	1. Select option 8 2. Enter MariaDB root password 3. MariaDB starts on internal network: mariadb-network 4. Benches connect via: DB_HOST=mariadb-database, DB_PORT=3306
⚠ **Root password is stored in ~/gitops/mariadb.env — keep this file secure.**	

Option 9 — Setup Shared PostgreSQL Database	
Description	Creates ~/gitops/postgres.env and starts a shared PostgreSQL container on the internal postgres-network.
When to Use	Before deploying a bench that uses PostgreSQL as the database.

Steps	1. Select option 9 2. Enter PostgreSQL password 3. PostgreSQL starts on internal network: postgres-network 4. Benches connect via: DB_HOST=postgres-database, DB_PORT=5432
⚠ **Not all Frappe apps support PostgreSQL. Check compatibility first.**	

Option 10 — Restart Infrastructure Servers	
Description	Restarts Traefik, MariaDB, and/or PostgreSQL containers using existing .env files.
When to Use	After a server reboot. When infrastructure containers crash.
Steps	1. Select option 10 2. Tool checks ~/gitops for traefik.env, mariadb.env, postgres.env 3. Each found service is restarted automatically 4. Success/failure reported for each
⚠ **Only services with .env files in ~/gitops are restarted.**	

Option 11 — Drop Infrastructure Services	
Description	Stops and removes Traefik, MariaDB, or PostgreSQL containers. Optional volume removal.
When to Use	When decommissioning a server or reconfiguring infrastructure from scratch.
Steps	1. Select option 11 2. Choose which service(s) to drop (or all) 3. Choose whether to remove data volumes 4. Confirm the destructive action 5. Services are stopped and removed
⚠ ⚠ **Removing volumes deletes ALL database data permanently. Bench containers lose DB connectivity.**	

Live Bench & Sites

Option 12 — Deploy Frappe / ERPNext Bench	
Description	Full bench deployment wizard. Creates a project .env, generates a resolved compose YAML, and starts the bench stack.
When to Use	Setting up a new Frappe/ERPNext project on a server with Traefik and DB already running.
Steps	1. Select option 12 2. Enter project name (e.g. erpnext-one) 3. Choose database: MariaDB or PostgreSQL 4. Enter DB password, DB host and port 5. Enter Let's Encrypt email 6. Enter site domain(s) — comma-separated 7. Choose HTTPS (recommended for production) 8. Optionally choose a custom Docker image

	9. Compose YAML generated and saved to ~/gitops/<project>.yaml 10. Bench stack starts — create site next (option 13)
⚠ **Requires Traefik (opt 7) and DB (opt 8 or 9) to be running first.**	

Option 13 — Create a New Site	
Description	Creates a new Frappe site inside a running bench — sets up the database schema and creates the site folder.
When to Use	After deploying a bench to create the actual website.
Steps	1. Select option 13 2. Enter project name 3. Enter site domain (must match SITES in the bench .env) 4. Enter DB root password 5. Enter site admin password 6. Site is created, migrated and cache cleared automatically
⚠ **Site domain must match exactly what was set in SITES during bench deploy.**	

Option 14 — Install an App on a Site	
Description	Installs a Frappe app (e.g. erpnext, hrms, payments) on an existing site and runs migrations.
When to Use	After site creation to add more modules. When a client needs a new app.
Steps	1. Select option 14 2. Enter project name 3. Enter site name 4. Enter app name (e.g. erpnext, hrms, payments) 5. App installs and migrate runs automatically
⚠ **The app must already be present inside the Docker image. Rebuild the image if not.**	

Option 15 — Uninstall an App from a Site	
Description	Removes a Frappe app and ALL its data (doctypes, records, files) from a site. Runs migrate after.
When to Use	When an app is no longer needed and its data can be deleted.
Steps	1. Select option 15 2. Enter project, site, and app name 3. Read the destructive warning carefully 4. Confirm (default is No) 5. App is uninstalled and migrate runs automatically
⚠ ⚠ **ALL app data is permanently deleted. Take a backup first.**	

Live Site Operations

Option 16 — Migrate Site

Description	Runs bench migrate — applies all pending database schema changes from new app or Frappe versions.
When to Use	After updating the Docker image. After installing or uninstalling any app.
Steps	1. Select option 16 2. Enter project name 3. Enter site name 4. Migration runs — may take several minutes for large sites 5. Success message shown when complete

⚠ **Always run migrate after an image update. If migration fails, check View Logs (opt 28).**

Option 17 — Clear Site Cache

Description	Clears Redis cache and website cache for a site. Fixes stale UI, missing assets, and config not showing in browser.
When to Use	After any deploy or config change when the UI looks broken or stale.
Steps	1. Select option 17 2. Enter project name 3. Enter site name 4. Both clear-cache and clear-website-cache run 5. Safe — no data is lost

⚠ **Safe to run at any time. Users may see brief slowdown on next load.**

Option 18 — Set Maintenance Mode

Description	Puts a site in maintenance mode (shows maintenance page to all visitors) or brings it back online.
When to Use	Before running a long migration, restore, or update that requires downtime.
Steps	1. Select option 18 2. Enter project and site name 3. Choose: Enable (site offline) or Disable (site online) 4. Status confirmed

⚠ ⚠ **Always DISABLE after maintenance is complete. Admins can still log in during maintenance.**

Option 19 — Enable / Disable Scheduler

Description	Enables or disables the Frappe background job scheduler. Scheduler handles email, auto-backup, and scheduled tasks.
When to Use	Disable before major maintenance to prevent background jobs interfering. Always re-enable after.
Steps	1. Select option 19 2. Enter project and site name 3. Choose: Enable or Disable 4. Confirmation shown

⚠ ⚠ **Always re-enable after maintenance. If disabled, emails and scheduled tasks do not run.**

Option 20 — Drop / Delete a Site

Description	Permanently deletes a Frappe site — removes its database, files, and all data from the bench.
When to Use	When decommissioning a site or removing a client environment.
Steps	1. Select option 20 2. Enter project and site name 3. Enter DB root password 4. Read the warning — confirm with Yes (default is No) 5. Site database and files are permanently removed
⚠ ⚠ **PERMANENT — all data deleted immediately. Take a backup first.**	

Option 21 — Restore Site from Backup	
Description	Restores a Frappe site from a .sql.gz backup file inside the backend container.
When to Use	After data loss. When migrating a site to a new server. Rolling back a bad migration.
Steps	1. Select option 21 2. Enter project and site name (site must already exist) 3. Enter backup file path inside container: e.g. sites/mysite/private/backups/20240101_db.sql.gz 5. Enter DB root password and new admin password 6. Restore runs — run Migrate Site (opt 16) after
⚠ **Site must exist before restoring. Run Migrate after restore.**	

Images

Option 22 — View Docker Images	
Description	Lists Docker images on the local machine with repo, tag, ID, size, and age. Option to remove an image.
When to Use	Before deploying — to check available images. To free disk space.
Steps	1. Select option 22 2. Choose: Frappe images only / All images / Search by name 3. Image list is displayed 4. Option to remove an image by ID or name:tag
⚠ **Cannot remove an image used by a running container.**	

Option 23 — Create Custom Image	
Description	Builds a custom Docker image with your chosen Frappe apps baked in using apps.json.
When to Use	When you need apps not in the official image (custom apps, hrms, payments, etc.).
Steps	1. Select option 23 2. Enter image name and tag (e.g. mycompany/frappe:v15.1.0) 3. Enter Frappe branch and git URL

	4. Choose build type: custom (full build) or layered (faster) 5. Build apps.json — saved to development/apps.json (inside the repo) 5a. Enter Python version (default: 3.12.4) and Node version (default: 18.17.1) 6. Build command: sudo DOCKER_BUILDKIT=1 docker build --no-cache 6a. Passes both --secret id=apps_json,src=development/apps.json AND --build-arg=APPS_JSON_BASE64=<base64> for compatibility with old and new Containerfile versions 7. Choose single or multi-platform (amd64 + arm64) 8. Optionally push to registry after build
⚠ **Full build takes 10–30 minutes. Requires internet access.**	

Option 24 — Update Image with Latest Git Code	
Description	Rebuilds an existing image with --no-cache so all apps are freshly cloned from git.
When to Use	When code has been pushed to git and you need a new build without changing the tag.
Steps	1. Select option 24 2. Existing local Frappe images are listed 3. Choose image to rebuild 4. Choose: keep same tag or use a new tag 5. Build runs with --no-cache — all layers rebuild 6. Optionally push to registry after build
⚠ **--no-cache means all layers rebuild — slower than a normal build.**	

Management

Option 25 — Update Bench (Pull → Restart → Migrate)	
Description	Complete 4-step bench update: pulls latest image, patches compose YAML, restarts services, migrates all sites.
When to Use	When a new image version is ready and a running production bench needs updating.
Steps	1. Select option 25 2. Enter project name 3. Enter new image name:tag (shows current image) 4. Step 1 — Image pulled from registry 5. Step 2 — Compose YAML patched with new image 6. Step 3 — Backend, workers, scheduler restarted with new image 7. Step 4 — All sites migrated and cache cleared 8. Success message with image name shown
⚠ **Test on staging first. Frontend stays up during worker restart.**	

Option 26 — Stop a Bench

Description	Stops all containers in a bench project. Volumes and all data are preserved.
When to Use	Before server maintenance. When a bench is not needed temporarily.
Steps	1. Select option 26 2. Enter project name 3. Bench containers stop 4. Data is intact — restart anytime with Update Bench or docker compose up
⚠ **Traefik will return 502 for that bench's domains while stopped.**	

Option 27 — View Running Containers

Description	Shows all running Docker containers with name, status, and port mappings.
When to Use	Quick health check of all services. After a deploy to confirm everything started.
Steps	1. Select option 27 2. Table shows: Name | Status | Ports
⚠ **Shows ALL Docker containers on the machine, not just Frappe ones.**	

Option 28 — View Container Logs

Description	Streams the last N lines of logs from any compose project's containers. Useful for debugging.
When to Use	When debugging errors. After a deploy to watch for startup problems.
Steps	1. Select option 28 2. Enter project name (or 'traefik', 'mariadb', 'postgres') 3. Enter service name (leave empty for all services) 4. Enter number of lines to show (default 50) 5. Logs are printed to screen
⚠ **For live log streaming use: docker compose logs -f <service>**	

Option 29 — Backup All Sites

Description	Runs bench backup --with-files on all sites in a bench. Saves DB dumps and private files inside the sites volume.
When to Use	Before any major update, migration, or image change. Regular scheduled backups.
Steps	1. Select option 29 2. Enter project name 3. Backup runs for all sites 4. Files saved to: sites/<site>/private/backups/ inside the volume
⚠ **Backups stay inside the Docker volume. Use Push to S3 (opt 30) for off-site storage.**	

Option 30 — Push Backup to S3 Storage

Description	Backs up all sites and syncs the backup files to an S3-compatible bucket (AWS, MinIO, Backblaze B2, etc.).
When to Use	For off-site backup storage. Compliance and disaster recovery.
Steps	1. Select option 30 2. Enter project name 3. Enter S3 bucket name and region 4. Enter AWS access key and secret key 5. Enter endpoint URL (leave blank for AWS, enter URL for MinIO etc.) 6. Enter S3 folder prefix (e.g. frappe-backups/myproject) 7. Backup runs locally, then files sync to S3

⚠ **AWS CLI is installed temporarily inside the container. Credentials are not stored.**

Option 31 — Bench Console (Python Shell)

Description	Opens an interactive Python shell inside the Frappe backend container with full access to the frappe API.
When to Use	Debugging data issues. Running one-off scripts. Inspecting or fixing documents without the UI.
Steps	1. Select option 31 2. Enter project name 3. Enter site name 4. Python shell opens in the terminal 5. Use frappe API: frappe.get_doc(), frappe.db.get_value(), etc. 6. Type exit() or press Ctrl+D to quit

⚠ ⚠ **Changes are LIVE and affect the production database. Always frappe.db.commit() or rollback.**

Option 32 — Clean Volumes, Networks & Build Cache

Description	Prunes unused Docker volumes, networks, and/or build cache. Shows disk usage before and after.
When to Use	When disk space is running low. After dropping old benches. After many image builds.
Steps	1. Select option 32 2. Current Docker disk usage shown 3. Choose what to clean: 1 = Volumes only 2 = Networks only 3 = Build cache only 4 = Volumes + Networks + Cache (full clean, default) 5 = Everything + unused images 9. If volumes selected and pwd.yml is running → offered to stop it first 10. Confirm and execute 11. Disk usage shown after cleanup

⚠ ⚠ **Removing volumes deletes ALL data in them. Only unused volumes are removed.**

5. Common Workflows

Fresh Server Setup — Production [Options: 1 → 2 → 7 → 8 → 12 → 13 → 14]	
Steps	1. Install Docker (opt 1) 2. Set repo (opt 2) 3. Setup Traefik with HTTPS (opt 7) — domain DNS must already point here 4. Setup shared MariaDB (opt 8) 5. Deploy bench (opt 12) — enter site domain, DB info, email 6. Create site (opt 13) 7. Install apps (opt 14) — e.g. erpnext, hrms
Note: Ensure DNS is pointed to this server before starting Traefik.	

Local Testing / Demo [Options: 1 → 2 → 3]	
Steps	1. Install Docker (opt 1) 2. Set repo (opt 2) 3. Local deploy via pwd.yml (opt 3) — choose image, site name, app 4. Access at http://localhost:8080 — Login: Administrator / admin 5. Site creation takes ~2–3 minutes on first run
Note: No Traefik or domain needed. All services on port 8080.	

Update Running Bench to New Image [Options: 25]	
Steps	1. Build or pull new image 2. Run: Update bench (opt 25) 3. Enter project name and new image name:tag 4. Tool runs automatically: pull → patch YAML → restart → migrate all sites 5. Clear cache runs automatically after migrate
Note: Always test on local/staging before updating production.	

Planned Maintenance Update (Safe) [Options: 18 → 19 → 29 → 25 → 19 → 18]	
Steps	1. Enable maintenance mode (opt 18) — site shows maintenance page 2. Disable scheduler (opt 19) — pause background jobs 3. Backup all sites (opt 29) — safety net 4. Update bench (opt 25) — pull new image and migrate 5. Re-enable scheduler (opt 19) 6. Disable maintenance mode (opt 18) — site back online 7. Verify site loads correctly in browser
Note: This is the safest production update process.	

Fix Internal Server Error After Deploy [Options: 28 → 16 → 17]	
Steps	1. View container logs (opt 28) to identify the error

	2. If 'Module not found' or migration needed → Migrate site (opt 16) 3. After migrate → Clear cache (opt 17) 4. Refresh the browser 5. If 'site already exists' in local → use Status & Diagnostics (opt 4)

Note: Most post-deploy errors are fixed by migrate + clear-cache.

Migrate Site to New Server [Options: 29 → 30 → 7 → 8 → 12 → 13 → 21 → 16]

Steps	1. On OLD server: Backup all sites (opt 29) 2. On OLD server: Push backup to S3 (opt 30) 3. On NEW server: Install Docker (opt 1), set repo (opt 2) 4. On NEW server: Setup Traefik (opt 7) and MariaDB (opt 8) 5. On NEW server: Deploy bench (opt 12) with same domain 6. On NEW server: Create blank site (opt 13) 7. On NEW server: Restore backup (opt 21) from S3 backup file 8. On NEW server: Migrate site (opt 16) 9. Update DNS to new server IP 10. Verify site works, then decommission old server

Note: DNS cutover should happen after restore is verified.

Decommission a Site [Options: 18 → 29 → 30 → 20]

Steps	1. Enable maintenance mode (opt 18) 2. Take final backup (opt 29) 3. Push backup to S3 for archival (opt 30) 4. Drop site (opt 20) — permanent 5. Verify the site no longer loads

Note: ⚠ Drop site is permanent. Confirm backup is safe before dropping.

Debug Site Issues with Console [Options: 31]

Steps	1. Open bench console (opt 31) 2. Enter project and site name 3. Python shell opens — examples: frappe.get_doc('User', 'Administrator') frappe.db.get_value('Company', 'MyCompany', 'default_currency') frappe.db.commit() ← always commit changes exit() ← to quit

Note: ⚠ Changes are live in the production database.

Free Up Disk Space [Options: 32]

Steps	1. Run Clean Docker (opt 32) 2. View current disk usage breakdown 3. Choose option 4 (full clean) or 5 (full clean + unused images) 4. If pwd.yml is running and volumes selected → offered to stop it first

	5. Confirm — unused volumes, networks, build cache removed 6. View reclaimed space
Note: Stop unused benches first to maximise space reclaimed.	

6. Quick Reference Card

#	Option Name	One-Line Description	Key Warning
1	**Install Docker**	Install Docker engine + Compose	
2	**Set Active Repo**	Navigate to or clone frappe_docker repo	
3	**Local Deploy**	Full local stack on port 8080	
4	**Local Status**	Check health + auto-diagnose errors	
5	**Stop Local**	Stop local stack (data kept)	
6	**Drop Local**	Remove local stack (3 levels)	
7	**Setup Traefik**	Start Traefik with Let's Encrypt	**DNS must point here**
8	**Setup MariaDB**	Start shared MariaDB container	
9	**Setup PostgreSQL**	Start shared PostgreSQL container	**Check app compat.**
10	**Restart Infra**	Restart Traefik/MariaDB/PostgreSQL	
11	**Drop Infra**	Remove infrastructure containers	**Data loss risk**
12	**Deploy Bench**	Create bench YAML + start containers	**Needs Traefik + DB**
13	**Create Site**	Create new Frappe site in bench	
14	**Install App**	Install app on site + migrate	**App must be in image**
15	**Uninstall App**	Remove app + all its data	**IRREVERSIBLE**
16	**Migrate Site**	bench migrate — sync DB schema	**Run after every update**
17	**Clear Cache**	Clear Redis + website cache	
18	**Maintenance Mode**	Toggle site offline / online	**Re-enable after work**
19	**Scheduler**	Enable or disable background jobs	**Re-enable after work**
20	**Drop Site**	Permanently delete site + DB	**IRREVERSIBLE**
21	**Restore Backup**	Restore .sql.gz into site	**Run migrate after**
22	**View Images**	List / remove local Docker images	
23	**Create Image**	Build image with apps.json	**10–30 min build time**
24	**Update Image**	Rebuild image with --no-cache	**All layers rebuild**
25	**Update Bench**	Pull → patch → restart → migrate	**Test on staging first**
26	**Stop Bench**	Stop bench containers (data kept)	
27	**View Containers**	docker ps — all running services	
28	**View Logs**	Stream container logs	
29	**Backup Sites**	bench backup --with-files all sites	**Copy off-server**
30	**Push to S3**	Backup + sync to S3 bucket	
31	**Bench Console**	Python shell in backend container	**Changes are LIVE**

32	**Clean Docker**	Prune volumes/networks/cache	**Volume = data loss**

7. Troubleshooting Guide

Problem: Internal Server Error in browser	
Fix	1. Run View Logs (opt 28) on the backend service 2. If 'Module not found' → Run Migrate site (opt 16) 3. If migration passes → Run Clear cache (opt 17) 4. If still broken → Check if create-site completed successfully

Problem: 'Site already exists' during deploy	
Fix	1. Old volumes from a previous deploy still exist 2. Use Local Deploy Status & Diagnostics (opt 4) for local stack 3. Choose Fresh deploy to wipe old volumes and redeploy cleanly 4. Or choose Migrate to keep data and sync the new image

Problem: 502 Bad Gateway from Traefik	
Fix	1. The bench containers have stopped or crashed 2. Run View Containers (opt 27) to check container status 3. Run Restart Infrastructure (opt 10) if Traefik itself is down 4. Run View Logs (opt 28) on the frontend or backend service

Problem: Docker not found / not installed	
Fix	1. Run Install Docker (opt 1) 2. After install, re-run the script 3. On WSL2, ensure Docker Desktop is running on Windows

Problem: Image pull fails	
Fix	1. Check internet connectivity 2. Run: docker login if the registry requires authentication 3. Verify the image name and tag are correct (opt 22 to list local images)

Problem: Migration fails with error	
Fix	1. Run View Logs (opt 28) — backend service — to see the full traceback 2. Common causes: missing app in image, incompatible app versions 3. Rebuild the image with the correct app versions (opt 23 or 24) 4. Deploy the new image (opt 25) and run migrate again (opt 16)

Problem: Disk space is full

Fix	1. Run Clean Docker (opt 32) — option 4 (full clean) or 5 (+ images) 2. Stop unused benches first (opt 26) to free up volume space 3. Move old backups out of the volume or push to S3 (opt 30)

Problem: Scheduler / emails not working	
Fix	1. Check if scheduler is disabled: run Enable/Disable Scheduler (opt 19) 2. Enable the scheduler for the affected site 3. Check scheduler container logs (opt 28 → scheduler service) 4. Run bench doctor inside console (opt 31) for full diagnostics